DBMS WEEKLY TASK – SALES AND CUSTOMER ANALYTICS SYSTEM

Project Title: IKEA E-Commerce Database Management System
DBMS Task Title: Sales and Customer Analytics System
Subject: Database Management Systems
Week: 9
Database: Oracle SQL*Plus 21c XE

Objectives

1. Apply COUNT(), SUM(), AVG(), MIN(), and MAX().
2. Generate total sales reports.
3. Find top customers based on purchase amount.
4. Identify best-selling products.
5. Perform category-wise sales analysis.

*Note: the output images in this record are illustrative examples prepared from sample customer and order data. They are not screenshots from the live database. Replace them with your own SQL*Plus screenshots before submission.*

SCHEMA VERIFICATION

Run these commands first and compare the column names with the queries in this record. The Product and Order_Item columns are known. The queries assume Customer(Customer_ID, Customer_Name) and an order table with Order_ID, Customer_ID, Order_Date and Total_Amount; these need to be verified, and the queries in Sections 1, 2, 3 and 6 must be changed if your table differs. ORDER is a reserved word in Oracle, so the order table is written as "Order" in double quotes. If your table is named Orders, use Orders instead.

```
DESC Customer;
DESC Product;
DESC "Order";
DESC Order_Item;
DESC Payment;
```

1. AGGREGATE FUNCTIONS

Aggregate functions perform a calculation on many rows and return a single result.

1.1 COUNT()

COUNT() returns the number of rows. Here it gives the total number of orders.

```
SELECT COUNT(*) AS Total_Orders
FROM "Order";
```

```
SQL> SELECT COUNT(*) AS Total_Orders
  2  FROM "Order";

TOTAL_ORDERS
------------
           9

1 row selected.

SQL>
```

1.2 SUM()

SUM() adds the values of a column. Adding the order amounts gives the total sales.

```
SELECT SUM(Total_Amount) AS Total_Sales
FROM "Order";
```

```
SQL> SELECT SUM(Total_Amount) AS Total_Sales
  2  FROM "Order";

TOTAL_SALES
-----------
      92276

1 row selected.

SQL>
```

1.3 AVG()

AVG() gives the average order amount, rounded to two decimal places.

```
SELECT ROUND(AVG(Total_Amount), 2) AS Average_Order_Amount
FROM "Order";
```

```
SQL> SELECT ROUND(AVG(Total_Amount), 2) AS Average_Order_Amount
  2  FROM "Order";

AVERAGE_ORDER_AMOUNT
--------------------
            10252.89

1 row selected.

SQL>
```

1.4 MIN()

MIN() returns the smallest order amount.

```
SELECT MIN(Total_Amount) AS Lowest_Order_Amount
FROM "Order";
```

```
SQL> SELECT MIN(Total_Amount) AS Lowest_Order_Amount
  2  FROM "Order";

LOWEST_ORDER_AMOUNT
-------------------
               4097

1 row selected.

SQL>
```

1.5 MAX()

MAX() returns the largest order amount.

```
SELECT MAX(Total_Amount) AS Highest_Order_Amount
FROM "Order";
```

```
SQL> SELECT MAX(Total_Amount) AS Highest_Order_Amount
  2  FROM "Order";

HIGHEST_ORDER_AMOUNT
--------------------
               18393

1 row selected.

SQL>
```

2. TOTAL SALES REPORTS

Total sales are calculated by adding the order amounts with SUM(). GROUP BY is used when the total is needed for each customer.

The overall sales report gives the number of orders and the total sales generated by all of them.

```
SELECT COUNT(*) AS Total_Orders,
SUM(Total_Amount) AS Total_Sales
FROM "Order";
```

```
SQL> SELECT COUNT(*) AS Total_Orders,
  2  SUM(Total_Amount) AS Total_Sales
  3  FROM "Order";

TOTAL_ORDERS TOTAL_SALES
------------ -----------
           9       92276

1 row selected.

SQL>
```

2.1 Customer-wise Sales Report

Each customer is joined with their orders, the amounts are added for each customer, and the report is sorted from the highest purchase amount to the lowest.

```
SELECT c.Customer_ID, c.Customer_Name,
SUM(o.Total_Amount) AS Total_Purchase
FROM Customer c
JOIN "Order" o
ON c.Customer_ID = o.Customer_ID
GROUP BY c.Customer_ID, c.Customer_Name
ORDER BY Total_Purchase DESC, c.Customer_ID;
```

```
SQL> SELECT c.Customer_ID, c.Customer_Name,
  2  SUM(o.Total_Amount) AS Total_Purchase
  3  FROM Customer c
  4  JOIN "Order" o
  5  ON c.Customer_ID = o.Customer_ID
  6  GROUP BY c.Customer_ID, c.Customer_Name
  7  ORDER BY Total_Purchase DESC, c.Customer_ID;

CUSTOMER_ID CUSTOMER_NAME TOTAL_PURCHASE
----------- ------------ --------------
       1004 Meena Priya            22296
       1001 Arun Kumar             18996
       1005 Suresh Babu            18393
       1002 Divya Lakshmi          16096
       1003 Karthik Raja            9997
       1006 Anitha Devi             6498

6 rows selected.

SQL>
```

3. TOP CUSTOMERS BASED ON PURCHASE AMOUNT

The customers are ranked by the total amount they have spent, using SUM(), GROUP BY and ORDER BY.

3.1 Customer Ranking

```
SELECT c.Customer_ID, c.Customer_Name,
COUNT(o.Order_ID) AS Total_Orders,
SUM(o.Total_Amount) AS Total_Purchase
FROM Customer c
JOIN "Order" o
ON c.Customer_ID = o.Customer_ID
GROUP BY c.Customer_ID, c.Customer_Name
ORDER BY Total_Purchase DESC, c.Customer_ID;
```

```
SQL> SELECT c.Customer_ID, c.Customer_Name,
  2  COUNT(o.Order_ID) AS Total_Orders,
  3  SUM(o.Total_Amount) AS Total_Purchase
  4  FROM Customer c
  5  JOIN "Order" o
  6  ON c.Customer_ID = o.Customer_ID
  7  GROUP BY c.Customer_ID, c.Customer_Name
  8  ORDER BY Total_Purchase DESC, c.Customer_ID;

CUSTOMER_ID CUSTOMER_NAME TOTAL_ORDERS TOTAL_PURCHASE
----------- ------------ ------------ --------------
       1004 Meena Priya             2          22296
       1001 Arun Kumar              2          18996
       1005 Suresh Babu             1          18393
       1002 Divya Lakshmi           2          16096
       1003 Karthik Raja            1           9997
       1006 Anitha Devi             1           6498

6 rows selected.

SQL>
```

3.2 Top Five Customers

FETCH FIRST 5 ROWS ONLY keeps the first five rows after the sorting is done.

```
SELECT c.Customer_ID, c.Customer_Name,
SUM(o.Total_Amount) AS Total_Purchase
FROM Customer c
JOIN "Order" o
ON c.Customer_ID = o.Customer_ID
GROUP BY c.Customer_ID, c.Customer_Name
ORDER BY Total_Purchase DESC, c.Customer_ID
FETCH FIRST 5 ROWS ONLY;
```

```
SQL> SELECT c.Customer_ID, c.Customer_Name,
  2  SUM(o.Total_Amount) AS Total_Purchase
  3  FROM Customer c
  4  JOIN "Order" o
  5  ON c.Customer_ID = o.Customer_ID
  6  GROUP BY c.Customer_ID, c.Customer_Name
  7  ORDER BY Total_Purchase DESC, c.Customer_ID
  8  FETCH FIRST 5 ROWS ONLY;

CUSTOMER_ID CUSTOMER_NAME TOTAL_PURCHASE
----------- ------------- --------------
       1004 Meena Priya            22296
       1001 Arun Kumar             18996
       1005 Suresh Babu            18393
       1002 Divya Lakshmi          16096
       1003 Karthik Raja            9997

5 rows selected.

SQL>
```

4. BEST-SELLING PRODUCTS

The quantity sold for each product is found with SUM(Quantity) after joining Product and Order_Item on Product_ID.

```
SELECT p.Product_ID, p.Product_Name,
SUM(oi.Quantity) AS Total_Quantity
FROM Product p
JOIN Order_Item oi
ON p.Product_ID = oi.Product_ID
GROUP BY p.Product_ID, p.Product_Name
ORDER BY Total_Quantity DESC, p.Product_ID;
```

```
SQL> SELECT p.Product_ID, p.Product_Name,
  2  SUM(oi.Quantity) AS Total_Quantity
  3  FROM Product p
  4  JOIN Order_Item oi
  5  ON p.Product_ID = oi.Product_ID
  6  GROUP BY p.Product_ID, p.Product_Name
  7  ORDER BY Total_Quantity DESC, p.Product_ID;

PRODUCT_ID PRODUCT_NAME           TOTAL_QUANTITY
---------- ---------------------- --------------
       110 FEJKA Artificial Plant              5
       106 INGOLF Dining Chair                 4
       103 LACK Coffee Table                   3
       107 FADO Table Lamp                     3
       101 KALLAX Shelf Unit                   2
       102 MALM Bed Frame                      2
       108 MULIG Clothes Rack                  2
       104 POANG Armchair                      1
       105 MICKE Desk                          1
       109 APPLARO Outdoor Table               1

10 rows selected.

SQL>
```

4.1 Product Sales Amount

The sales amount is taken from Total_Price in Order_Item, which holds the price actually charged in each order after the discount, instead of the current Price in the Product table.

```
SELECT p.Product_ID, p.Product_Name,
SUM(oi.Total_Price) AS Total_Sales
FROM Product p
JOIN Order_Item oi
ON p.Product_ID = oi.Product_ID
GROUP BY p.Product_ID, p.Product_Name
ORDER BY Total_Sales DESC, p.Product_ID;
```

```
SQL> SELECT p.Product_ID, p.Product_Name,
  2  SUM(oi.Total_Price) AS Total_Sales
  3  FROM Product p
  4  JOIN Order_Item oi
  5  ON p.Product_ID = oi.Product_ID
  6  GROUP BY p.Product_ID, p.Product_Name
  7  ORDER BY Total_Sales DESC, p.Product_ID;

PRODUCT_ID PRODUCT_NAME           TOTAL_SALES
---------- ---------------------- -----------
       102 MALM Bed Frame               24498
       106 INGOLF Dining Chair          15996
       101 KALLAX Shelf Unit             9998
       109 APPLARO Outdoor Table         8999
       103 LACK Coffee Table             7497
       104 POANG Armchair                6999
       105 MICKE Desk                    5999
       107 FADO Table Lamp               4297
       108 MULIG Clothes Rack            3998
       110 FEJKA Artificial Plant        3995

10 rows selected.

SQL>
```

5. CATEGORY-WISE SALES ANALYSIS

Category-wise analysis shows which product categories bring in the most sales, which helps in deciding what to stock and promote. The category name column of the Category table is not confirmed, so Category_ID is used.

```
SELECT p.Category_ID,
SUM(oi.Quantity) AS Total_Quantity,
SUM(oi.Total_Price) AS Total_Sales
FROM Product p
JOIN Order_Item oi
ON p.Product_ID = oi.Product_ID
GROUP BY p.Category_ID
ORDER BY Total_Sales DESC, p.Category_ID;
```

```
SQL> SELECT p.Category_ID,
  2  SUM(oi.Quantity) AS Total_Quantity,
  3  SUM(oi.Total_Price) AS Total_Sales
  4  FROM Product p
  5  JOIN Order_Item oi
  6  ON p.Product_ID = oi.Product_ID
  7  GROUP BY p.Category_ID
  8  ORDER BY Total_Sales DESC, p.Category_ID;

CATEGORY_ID TOTAL_QUANTITY TOTAL_SALES
----------- -------------- -----------
          2              2       24498
          4              4       15996
          1              4       14496
          7              4       13996
          6              1        8999
          5              1        5999
          8              3        4297
         10              5        3995

8 rows selected.

SQL>
```

6. COMBINED BUSINESS SALES REPORT

This report joins Customer, Order, Order_Item and Product, and shows one row for every product in every order. Category_ID is read from Product. It shows what each customer bought and in what quantity and amount, which helps in understanding customer purchases and product sales together.

```
SELECT c.Customer_ID AS Cust_ID, c.Customer_Name,
o.Order_ID, p.Product_ID AS Prod_ID,
p.Product_Name, p.Category_ID AS Cat_ID,
oi.Quantity AS Qty, oi.Unit_Price,
oi.Total_Price
FROM Customer c
JOIN "Order" o
ON c.Customer_ID = o.Customer_ID
JOIN Order_Item oi
ON o.Order_ID = oi.Order_ID
JOIN Product p
ON oi.Product_ID = p.Product_ID
ORDER BY o.Order_ID, p.Product_ID;
```

```
SQL> SELECT c.Customer_ID AS Cust_ID, c.Customer_Name,
  2  o.Order_ID, p.Product_ID AS Prod_ID,
  3  p.Product_Name, p.Category_ID AS Cat_ID,
  4  oi.Quantity AS Qty, oi.Unit_Price,
  5  oi.Total_Price
  6  FROM Customer c
  7  JOIN "Order" o
  8  ON c.Customer_ID = o.Customer_ID
  9  JOIN Order_Item oi
 10  ON o.Order_ID = oi.Order_ID
 11  JOIN Product p
 12  ON oi.Product_ID = p.Product_ID
 13  ORDER BY o.Order_ID, p.Product_ID;

CUST_ID CUSTOMER_NAME ORDER_ID PROD_ID PRODUCT_NAME           CAT_ID QTY UNIT_PRICE TOTAL_PRICE
------- ------------- -------- ------- ---------------------- ------ --- ---------- -----------
   1001 Arun Kumar        5001     101 KALLAX Shelf Unit           7   1       4999        4999
   1001 Arun Kumar        5001     103 LACK Coffee Table           1   2       2499        4998
   1002 Divya Lakshmi     5002     102 MALM Bed Frame              2   1      12999       11999
   1003 Karthik Raja      5003     105 MICKE Desk                  5   1       5999        5999
   1003 Karthik Raja      5003     108 MULIG Clothes Rack          7   2       1999        3998
   1004 Meena Priya       5004     104 POANG Armchair              1   1       6999        6999
   1004 Meena Priya       5004     107 FADO Table Lamp             8   2       1499        2798
   1001 Arun Kumar        5005     109 APPLARO Outdoor Table       6   1       8999        8999
   1005 Suresh Babu       5006     106 INGOLF Dining Chair         4   4       3999       15996
   1005 Suresh Babu       5006     110 FEJKA Artificial Plant     10   3        799        2397
   1006 Anitha Devi       5007     101 KALLAX Shelf Unit           7   1       4999        4999
   1006 Anitha Devi       5007     107 FADO Table Lamp             8   1       1499        1499
   1002 Divya Lakshmi     5008     103 LACK Coffee Table           1   1       2499        2499
   1002 Divya Lakshmi     5008     110 FEJKA Artificial Plant     10   2        799        1598
   1004 Meena Priya       5009     102 MALM Bed Frame              2   1      12999       12499

15 rows selected.

SQL>
```

7. CONCLUSION

In this task, aggregate functions were used on the IKEA E-Commerce Database to count orders and calculate total, average, lowest and highest order amounts. With JOIN, GROUP BY and ORDER BY, the same functions gave customer-wise sales, the top customers, the best-selling products and the sales of each category. The combined report brings these details together for each order.